



**Hochschule
Hof**

University of
Applied Sciences

Software Engineering for Industrial Applications
Mandatory Internship SoSe-2026

**Development of
AI-powered Retrieval-Augmented
Generation (RAG) system
“Software Verification and Validation Report”**

Submitted To

Prof. Dr. Matthias Meitner
Hof University of Applied Sciences

Submitted By

Md Fahim Hossain
Student ID: 00154924

Table of Contents

1. Introduction & Company Overview	3
1.1. Introduction	3
1.2. Company Overview	3
1.3. Department Overview	4
1.4. Motivation for the Project	4
2. Role & Tasks	4
3. Project Overview	5
4. Tools & Frameworks	6
5. Code Review Process	7
5.1. Self-Review and Refactoring	8
5.2. Architectural Review and Code Quality Improvements	8
5.3. Final Review Before Feature Completion	9
6. Testing	10
6.1. Unit Testing	10
6.1.1. Testing the Document Chunking Service	10
6.1.2. Testing the PDF Extraction Service	11
6.1.3. Testing the Word Extraction Service	12
6.1.4. Testing the Excel Extraction Service	12
6.1.5. Testing the RAG Retrieval Service	13
6.2. API Testing	14
6.2.1. Testing the Embedding API Integration	14
6.2.2. Testing the Language Model API Integration	15
6.2.3. Testing API Failure Scenarios	16
6.3. Load Testing	17
6.3.1. Concurrent Document Processing	17
6.3.2. Concurrent Question Answering	18
6.3.3. Database Performance During Retrieval	18
6.3.4. Overall System Stability	18
6.4. Test Cases	19
7. Validation	24
7.1. Sprint Review	25

7.2. User Story Testing	28
7.3. Acceptance Testing	31
8. Conclusion	34
9. References	35

List of Tables

Table 1: TC01 – Document Upload Test Data	20
Table 2: TC02 – Document Processing Pipeline Test Data	21
Table 3: TC03 – AI-Assisted Question Answering Test Data	22
Table 4: TC04 – Multi-Document Retrieval Test Data	22
Table 5: TC05 – Conversation History Test Data	23
Table 6: TC06 – Document Deletion Test Data	24

List of Figures

Figure 1: Dispatching background document processing after a successful upload	8
Figure 2: Document processing with asynchronous embedding generation	9
Figure 3: Unit test verifying document chunk generation and metadata preservation	11
Figure 4: Unit test validating PDF text normalization	11
Figure 5: Unit test verifying Word document text extraction	12
Figure 6: Unit test verifying spreadsheet text extraction	13
Figure 7: Unit test verifying semantic document retrieval	13
Figure 8: Mocking the embedding API response	14
Figure 9: Verifying the embedding API request and response	15
Figure 10: Mocking the language model API response	15
Figure 11: Verifying prompt construction before the language model request	16
Figure 12: Verifying missing API credentials and unsupported provider configurations	16
Figure 13: Laravel queue worker processing documents during concurrent uploads	17

1. Introduction & Company Overview

1.1. Introduction

As part of the Master's program in Software Engineering for Industrial Applications at Hof University of Applied Sciences, students are required to apply theoretical knowledge in a practical industrial environment through project-based work. This report documents the software verification and validation activities performed during the development of an AI-powered Retrieval-Augmented Generation (RAG) system for enterprise document management and knowledge retrieval.

Modern organizations generate and maintain large volumes of technical documents, reports, manuals, process descriptions, and operational guidelines. Accessing relevant information from these documents often requires significant manual effort, leading to reduced efficiency and increased time spent searching for knowledge. To address this challenge, an intelligent document retrieval system was designed and developed that combines document processing, semantic search, and Artificial Intelligence technologies to provide context-aware answers based on organizational documents.

The project was developed during my work as a Working Student at Siemens Energy. The development process involved requirements analysis, software specification, system design, architecture planning, implementation, and validation of the proposed solution. This report focuses on the verification and validation activities performed during development. It describes the code review process, unit testing, API testing, performance testing, functional test cases, and validation activities carried out to ensure that the developed system satisfies its specified requirements and operates reliably under practical conditions.

The objective of the project was to develop a scalable and maintainable software solution that can process organizational documents, retrieve relevant information, and help users find answers through natural language queries.

1.2. Company Overview

Siemens Energy is one of the world's leading energy technology companies, operating across the entire energy value chain. The company develops innovative solutions for power generation, transmission, storage, and industrial applications while supporting the global transition toward sustainable and reliable energy systems.

With operations in more than 90 countries and a workforce of approximately 100,000 employees worldwide, Siemens Energy serves customers in both conventional and renewable energy sectors. The company provides technologies and services for gas turbines, steam turbines, power generation systems, grid infrastructure, hydrogen solutions, industrial applications, and digital transformation initiatives.

Digitalization plays a critical role in Siemens Energy's strategy. The company continuously invests in advanced software solutions, data-driven decision making, artificial intelligence, cloud technologies, and process automation to improve operational efficiency and support business innovation. These initiatives enable employees and stakeholders to access information more effectively, optimize workflows, and enhance organizational productivity.

1.3. Department Overview

The project was conducted within the SE GT DG OE department of Siemens Energy.

The department is involved in supporting digitalization initiatives, process optimization, and data-driven solutions that enhance operational efficiency across business functions. Various software tools, reporting systems, and analytical platforms are utilized to manage information, automate processes, and improve collaboration among stakeholders.

As part of ongoing digital transformation efforts, the department actively explores modern technologies such as cloud platforms, data analytics, artificial intelligence, and enterprise software systems. These technologies are used to streamline business processes, improve knowledge management, and support informed decision-making.

The development of the AI-powered document retrieval system aligns with these objectives by providing an intelligent mechanism for managing and accessing organizational knowledge. The project contributes to the broader goal of improving information accessibility and reducing the effort required to locate relevant content within large collections of enterprise documents.

1.4. Motivation for the Project

Organizations frequently store critical knowledge across numerous documents, including technical manuals, operational procedures, project documentation, and internal reports. Although these documents contain valuable information, retrieving specific content often requires manual searching, which can be time-consuming and inefficient.

The increasing adoption of Artificial Intelligence technologies has created opportunities to enhance document management systems through semantic understanding and context-aware information retrieval. Instead of relying solely on keyword-based searches, modern AI-assisted systems can understand user intent and identify relevant information based on contextual meaning.

The motivation behind this project was to design and develop a software solution capable of improving access to organizational knowledge while maintaining a user-friendly experience. By integrating document processing, semantic retrieval mechanisms, and AI-assisted response generation, the proposed system aims to support employees in obtaining accurate information quickly and efficiently.

The project also provided an opportunity to apply software engineering principles, including requirements engineering, system design, architectural modeling, and software development methodologies within a real industrial environment.

2. Role & Tasks

During my internship at Siemens Energy, I worked as a Working Student in the Digital Solutions team, contributing to both enterprise software development and data engineering activities. My responsibilities included supporting internal data analytics initiatives while leading the design and development of an AI-powered Retrieval-Augmented Generation (RAG) system for enterprise document management and knowledge retrieval. Throughout the internship, I collaborated with software

engineers and business stakeholders to analyse requirements, design software components, implement new features, and document the developed solution.

My primary responsibility was the development of the RAG system. I participated in the complete software development lifecycle, including requirements analysis, system design, backend development, user interface implementation, and technical documentation. The system was designed to provide secure and efficient access to enterprise knowledge through semantic search and AI-assisted question answering.

As part of this work, I developed the document management and processing workflow, enabling users to upload documents, monitor processing progress, and retrieve information through a conversational interface. I also implemented semantic retrieval, conversation management, and background processing features to support efficient document indexing and question answering. In addition, I contributed to improving the system architecture, optimizing database operations, and preparing the application for containerized deployment.

Alongside the RAG project, I supported enterprise data engineering activities involving Python, SQL, and Snowflake. My work included developing and validating data processing workflows, transforming business data for reporting purposes, and contributing to the development of Power BI dashboards in collaboration with domain experts.

Throughout the internship, I participated in technical discussions, code reviews, and documentation activities while collaborating with software engineers and business stakeholders.

The combination of enterprise software development, AI-assisted document retrieval, and data engineering provided valuable experience in designing scalable software systems, collaborating within multidisciplinary teams, and applying software engineering principles to solve practical industrial challenges.

3. Project Overview

The project developed during this internship is an AI-powered Retrieval-Augmented Generation (RAG) system designed to improve enterprise document management and knowledge retrieval. The application enables users to upload organizational documents, process them automatically, and interact with their contents through a conversational interface. Instead of manually searching through lengthy reports or technical documentation, users can ask questions in natural language and receive responses generated from the relevant information contained within the uploaded documents.

The primary objective of the system is to improve information accessibility while ensuring that responses are generated from trusted organizational documents rather than relying solely on the knowledge of a language model. By combining semantic document retrieval with AI-assisted response generation, the system provides context-aware, document-grounded answers while supplying only the most relevant document content for response generation.

Document Upload and Processing

Users can upload and manage enterprise documents through a centralized repository. After upload, the system automatically extracts document text, performs preprocessing, divides the content into semantic chunks, and prepares it for retrieval. This automated processing pipeline minimizes manual effort and ensures that documents become searchable once processing has been completed.

Semantic Retrieval and Question Answering

The system retrieves relevant document sections using semantic similarity rather than exact keyword matching, allowing users to ask questions in natural language. The retrieved document content is then provided to a language model, which generates context-aware responses grounded in the uploaded documents. The application also presents document references so that users can verify the origin of the retrieved information.

Conversation Management

The application provides a conversation-oriented interface where users can create multiple conversations, select one or more processed documents as knowledge sources, review previous discussions, and continue asking follow-up questions while preserving conversational context.

Overall, the developed RAG system combines automated document processing, semantic search, and AI-assisted question answering within a unified platform for enterprise knowledge retrieval. The solution improves information accessibility, reduces the effort required to locate relevant knowledge, and provides employees with an efficient and reliable way to interact with organizational documents.

4. Tools & Frameworks

The development of the AI-powered document retrieval system relied on a combination of backend technologies, development tools, testing utilities, and supporting frameworks. These tools were selected to support efficient software development, debugging, testing, and deployment throughout the project lifecycle.

Laravel Framework

Laravel 12 served as the primary backend framework for implementing the application. It provided the project structure, routing, authentication, middleware, database abstraction, queue management, and testing support. Its modular architecture enabled clean separation between business logic and presentation while improving maintainability and testability (Laravel, 2025).

PostgreSQL and pgvector

PostgreSQL was used as the primary relational database for storing application data, including users, uploaded documents, conversations, and metadata. The pgvector extension enabled efficient storage and similarity search of document embeddings, providing the foundation for semantic document retrieval (PostgreSQL Global Development Group, 2025; pgvector Contributors, 2025).

AI Models

The system uses two AI models during document processing and question answering. An embedding

model generates vector representations of document chunks, while a language model produces context-aware responses using the retrieved document content.

Poppler and Tesseract OCR

Poppler was used to extract text from digitally generated PDF documents, while Tesseract OCR processed scanned documents. Together, these tools enabled the system to support both text-based and scanned PDF documents (**Poppler Developers, 2025; Tesseract OCR Contributors, 2025**).

Queue Processing

Laravel's queue system was used to execute computationally intensive tasks such as document parsing, embedding generation, and indexing asynchronously. This approach improved application responsiveness by performing long-running operations in the background.

Git and GitHub

Git was used for version control throughout development, while GitHub served as the central repository for source code management, collaboration, and code review.

Docker

Docker was used to containerize the application and its supporting services, providing a consistent development and deployment environment across different machines (**Docker Inc., 2025**).

Development and API Testing Tools

Several development tools were used throughout implementation and verification:

- **Visual Studio Code** served as the primary integrated development environment for software development and debugging.
- **Postman** was used to test backend API endpoints, authentication, and request validation.
- **PgAdmin** was used to inspect database records, execute SQL queries, and manage PostgreSQL.
- **Browser Developer Tools** were used to inspect network requests, debug frontend interactions, and validate user interface behaviour.

Performance and Load Testing

Performance testing evaluated the application's behaviour under concurrent workloads, including document uploads, background processing, and question-answering requests. These tests helped verify application responsiveness and overall system stability under typical operating conditions.

5. Code Review Process

Software quality was continuously improved throughout the development of the AI-powered document retrieval system through regular code reviews performed during development (**Pressman & Maxim, 2020**). Instead of treating code review as a final activity before deployment, each completed feature was reviewed before integration into the main application. This iterative approach helped identify defects early, improve maintainability, and ensure consistency with the overall system architecture.

Since the project was developed incrementally, major features, including document management, document processing, semantic retrieval, conversation management, OCR support, and administrative

functionality, were reviewed after implementation. The reviews focused on code readability, correctness, maintainability, performance, and compliance with Laravel development practices.

The review activities were organized into three stages.

5.1. Self-Review and Refactoring

The first review stage was carried out immediately after implementing each feature. The objective was to improve code quality before integration by removing unnecessary complexity and ensuring that the implementation followed established coding practices.

The review focused on:

- Compliance with Laravel coding conventions.
- Clear naming of variables, methods, and classes.
- Proper separation of responsibilities between controllers, services, and background jobs.
- Removal of duplicated logic.
- Exception handling for potential runtime failures.
- Efficient database interaction.
- Overall readability and maintainability.

A key design decision throughout the project was to keep controllers responsible only for request validation, authorization, and dispatching background jobs, while computationally intensive operations such as document parsing, text chunking, and embedding generation were delegated to dedicated service classes and queue jobs.

For example, after a document upload is validated and stored, the controller simply dispatches a background processing job instead of executing the complete processing workflow synchronously.



```
ProcessDocumentJob::dispatch($document->id);

return redirect()
    ->route('documents.show', $document)
    ->with('success', 'Document uploaded successfully. Processing has started.')->go(f, seed, []);
}
```

Figure 1: Dispatching background document processing after a successful upload

This approach reduced controller complexity, improved code organization, and prevented long-running operations from blocking the user interface.

5.2. Architectural Review and Code Quality Improvements

After the initial implementation, the overall project structure was reviewed to ensure that newly developed components remained consistent with the intended architecture.

The architectural review concentrated on:

- Clear separation between business logic and presentation logic.

- Reusability of shared services.
- Consistent exception handling.
- Proper database transaction management.
- Efficient interaction with external services.

Several architectural improvements were introduced during development. Business logic was consolidated into reusable service classes, long-running operations were executed through queue jobs, common functionality was reused instead of duplicated, and database updates involving multiple operations were grouped into transactions.

The document processing workflow illustrates this design. After text extraction is completed successfully, database updates are executed within a transaction before the embedding generation job is dispatched.



```
DB.transaction(function () use ($document, $chunks) {

  // Store processed chunks

  $document->forceFill([
    'status' => Document::STATUS_CHUNKED,
    'total_chunks' => count($chunks),
  ]->save());

});

GenerateDocumentEmbeddingsJob::dispatch($document->id);
```

Figure 2: Document processing with asynchronous embedding generation

Using database transactions prevented inconsistent processing states, while separating embedding generation into an independent background job improved scalability and simplified testing of each processing stage.

5.3. Final Review Before Feature Completion

Before a feature was considered complete, a final review verified that it functioned correctly as part of the integrated application. The review ensured that newly implemented functionality behaved as expected under realistic usage scenarios and remained compatible with previously completed features.

The final review included verification of:

- Correct execution of application workflows.
- Database consistency after user actions.
- Queue execution and background processing.
- User interface behaviour and validation messages.

- Handling of exceptional conditions.
- Compatibility with existing functionality.

Particular attention was given to features involving multiple components, such as document upload, OCR processing, semantic indexing, and AI-assisted question answering. These workflows required validation of the complete processing pipeline rather than individual components alone.

Any issues identified during this stage were corrected before proceeding to the formal testing activities described in the following chapter.

6. Testing

Testing was carried out throughout the development of the AI-powered document retrieval system to verify the correctness, reliability, and robustness of the implemented software. The objective was to ensure that individual components functioned correctly, external integrations operated as expected, and the complete application satisfied its functional requirements before deployment (**International Organization for Standardization, 2013**). To achieve this, several testing approaches were adopted during development. Automated unit tests were implemented using Laravel's PHPUnit framework to validate individual services in isolation (**PHPUnit Contributors, 2025**). API testing verified communication with external AI services, while load testing and functional test cases evaluated the application's behaviour under realistic operating conditions.

Testing was performed incrementally as each major feature was completed. This approach simplified debugging, reduced regression issues, and ensured that newly implemented functionality integrated correctly with the existing system. The following sections describe the testing activities performed during the project.

6.1. Unit Testing

Unit testing focused on validating individual software components independently before they were integrated into the complete application. Laravel's built-in PHPUnit testing framework was used to implement automated tests located in the tests/Unit directory, enabling each service to be verified under controlled conditions without relying on other system components.

Each unit test targeted a specific responsibility of the corresponding service, making it easier to identify defects, perform regression testing after code changes, and maintain software quality throughout development.

The implemented unit tests covered the core document processing services, including document chunking, text extraction from supported document formats, and semantic retrieval. Together, these components form the foundation of the document processing pipeline and therefore required verification before integration with the remaining system.

6.1.1. Testing the Document Chunking Service

The DocumentChunker service was tested to verify that extracted document content is correctly divided into semantic chunks while preserving the metadata required for retrieval and citation generation.

The following test verifies that page information and extraction metadata are preserved during the chunking process.

```
$chunks = app(DocumentChunker::class)->chunkPages([
    ['page' => 5, 'content' => str_repeat('Page five sentence. ', 20)],
    ['page' => 6, 'content' => str_repeat('Page six sentence. ', 20)],
], chunkSize: 900, overlap: 100);

$this->assertNotEmpty($chunks);
$this->assertSame(5, $chunks[0]['page_start']);
$this->assertSame(6, $chunks[0]['page_end']);
$this->assertSame('pdf_text_extraction', $chunks[0]['metadata']['source']);
$this->assertSame(['native'], $chunks[0]['metadata']['extraction_methods']);
```

Figure 3: Unit test verifying document chunk generation and metadata preservation

Test Objectives

- Verify correct chunk generation.
- Confirm preservation of page boundaries.
- Validate extraction metadata.
- Verify document source identification.
- Ensure generated chunks remain suitable for semantic retrieval.

Expected Outcome

The generated chunks preserve document structure, page information, and extraction metadata, ensuring accurate semantic retrieval and citation generation.

Additional unit tests also verify OCR metadata preservation, spreadsheet metadata handling, and UTF-8 character integrity for different document types.

6.1.2. Testing the PDF Extraction Service

The PdfExtractorService was tested to verify that textual content extracted from PDF documents is correctly normalized before further processing. The following test validates the text normalization process.

```
$normalized = $normalize->invoke($service, $text);

$this->assertStringContainsString(
    'Invoice total: $1,250.00',
    $normalized
);

$this->assertStringNotContainsString(
    '%%%%%%%%',
    $normalized
);
```

Figure 4: Unit test validating PDF text normalization

Test Objectives

- Verify removal of extraction artifacts.
- Preserve meaningful document content.
- Validate UTF-8 encoding.
- Remove invalid control characters.
- Improve extracted text quality.

Expected Outcome

The normalization process removes unwanted characters while preserving readable document content for subsequent indexing and semantic retrieval.

6.1.3. Testing the Word Extraction Service

The WordExtractorService was tested to verify that textual content is extracted correctly from supported Microsoft Word documents and prepared for subsequent processing.

The following test validates successful Word document text extraction.



```
$pages = app(WordExtractorService::class)->extractPages($path);

$this->assertCount(1, $pages);
$this->assertNotNull($pages[0]['page']);
$this->assertSame('docx', $pages[0]['metadata']['source_type']);
$this->assertSame(
    'word_text_extraction',
    $pages[0]['metadata']['extraction_method']
);

$this->assertStringContainsString(
    'Mobile Purchase Notes',
    $pages[0]['content']
);
```

Figure 5: Unit test verifying Word document text extraction

Test Objectives

- Verify successful text extraction.
- Preserve document content.
- Validate extracted text format.
- Confirm compatibility with the document processing pipeline.

Expected Outcome

Text is extracted successfully from Word documents and prepared for semantic chunk generation without losing relevant information.

6.1.4. Testing the Excel Extraction Service

The ExcelExtractorService was tested to verify that spreadsheet data is converted into structured textual representations while preserving worksheet and row-level information.

The following test validates spreadsheet text extraction.

```

$pages = app(ExcelExtractorService::class)
    ->extractPages($path, 'xlsx');

$this->assertCount(1, $pages);

$this->assertStringContainsString(
    'Sheet: Invoices',
    $pages[0]['content']
);

$this->assertSame(
    'xlsx',
    $pages[0]['metadata']['source_type']
);

$this->assertSame(
    'spreadsheet_text_extraction',
    $pages[0]['metadata']['extraction_method']
);

```

Figure 6: Unit test verifying spreadsheet text extraction

Test Objectives

- Verify spreadsheet text extraction.
- Preserve worksheet metadata.
- Validate row information.
- Confirm compatibility with semantic indexing.

Expected Outcome

Spreadsheet contents are transformed into structured textual representations while preserving contextual metadata required during retrieval.

6.1.5. Testing the RAG Retrieval Service

The RagRetrievalService was tested to verify that the most relevant document chunks are retrieved based on semantic similarity before response generation. The following test validates semantic document retrieval.

```

config([
    'services.rag.top_k' => 6,
    'services.rag.summary_top_k' => 12,
]);

$service = new RagRetrievalService(
    Mockery::mock(EmbeddingService::class)
);

$this->assertSame(
    12,
    $service->limitForQuestion(
        'Summarize this document'
    )
);

```

Figure 7: Unit test verifying semantic document retrieval

Test Objectives

- Verify semantic retrieval accuracy.
- Confirm document filtering.
- Validate similarity ranking.
- Ensure correct retrieval of relevant document chunks.

Expected Outcome

The retrieval service consistently returns the most relevant document chunks for subsequent response generation while respecting the specified retrieval criteria.

6.2. API Testing

The AI-powered document retrieval system integrates with two external AI services: one for generating document embeddings and another for generating context-aware responses. API testing was performed to verify successful communication with these services, including request construction, response processing, and error handling.

Laravel's HTTP mocking functionality was used to simulate external API responses, allowing the integration to be tested without requiring network communication. This approach provided fast, repeatable, and deterministic test execution while validating both successful and failure scenarios.

The following sections describe the API testing activities performed during the project.

6.2.1. Testing the Embedding API Integration

The embedding service was tested to verify that document text is correctly submitted to the external embedding API and that the returned embedding vector is processed successfully.

The following test simulates the external API response using Laravel's HTTP mocking functionality.

```
Http::fake([
    'embedding-api.domain.com/*' => Http::response([
        'embedding' => [
            'values' => [0.1, -0.2, 0.3, 0.4, -0.5],
        ],
    ]),
]);
```

Figure 8: Mocking the embedding API response

Test Objectives

- Verify correct API request generation.
- Validate successful embedding generation.
- Confirm correct parsing of the returned vector.
- Verify embedding dimensions.
- Ensure no real network communication is required during testing.

Expected Outcome

The embedding service successfully processes the simulated API response and returns the expected embedding vector for further processing.



```
$this->assertSame([0.1, -0.2, 0.3, 0.4, -0.5], $embedding);

Http::assertSent(function ($request) {
    return $request->hasHeader('x-api-key', 'test-key')
        && $request['output_dimensionality'] === 5;
});
```

Figure 9: Verifying the embedding API request and response

After generating the embedding, the automated test also verifies both the returned values and the correctness of the outgoing HTTP request.

6.2.2. Testing the Language Model API Integration

The language model service was tested to verify correct prompt construction, communication with the external API, and processing of the generated response.

The external API was simulated using HTTP mocking to enable reliable and repeatable testing without depending on service availability.



```
Http::fake([
    'generative-ai.domain.com/*' => Http::response([
        'candidates' => [
            [
                'content' => [
                    'parts' => [
                        ['text' => 'Use shorter title tags [SEO Report, page 2].'],
                    ],
                ],
            ],
        ],
    ]),
]);
```

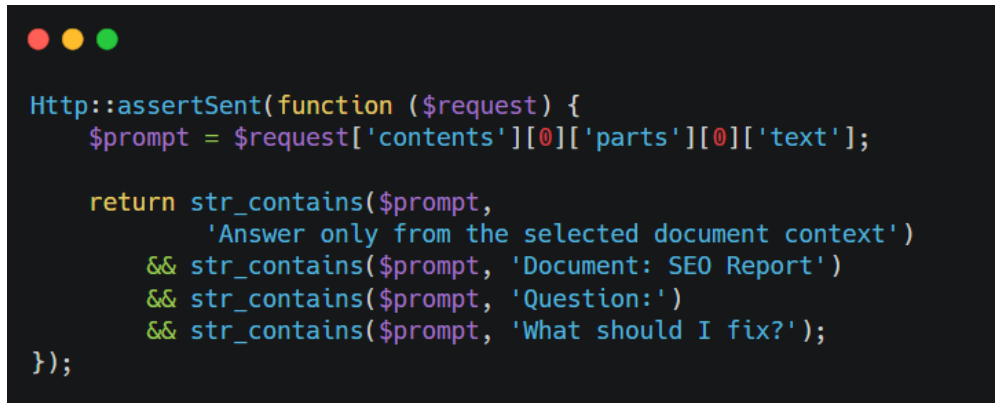
Figure 10: Mocking the language model API response

Test Objectives

- Verify successful API communication.
- Validate prompt construction.
- Confirm response parsing.
- Verify citation preservation.
- Ensure generated responses are returned correctly.

Expected Outcome

The language model returns a valid response based on the supplied document context, and the generated answer is successfully processed by the application.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in PHP and is a test function for an HTTP request. It uses `Http::assertSent` to verify that a request was sent with a specific prompt. The prompt is constructed from the request's contents, parts, and text. The function then returns a boolean result based on whether the prompt contains specific strings: 'Answer only from the selected document context', 'Document: SEO Report', 'Question:', and 'What should I fix?'.

```
Http::assertSent(function ($request) {  
    $prompt = $request['contents'][0]['parts'][0]['text'];  
  
    return str_contains($prompt,  
        'Answer only from the selected document context')  
        && str_contains($prompt, 'Document: SEO Report')  
        && str_contains($prompt, 'Question:')  
        && str_contains($prompt, 'What should I fix?');  
});
```

Figure 11: Verifying prompt construction before the language model request

The automated test also verifies that the generated prompt contains the required contextual information before being transmitted to the external service.

6.2.3. Testing API Failure Scenarios

In addition to successful API communication, the application was tested under invalid configuration scenarios to verify that failures are handled safely and predictably.

The following test verifies missing API credentials and unsupported provider configurations.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in PHP and is a test function for an application. It uses `$this->expectException` to expect a `RuntimeException` and `$this->expectExceptionMessage` to expect the message 'API key is missing'. The test then calls `app(EmbeddingService::class)->embedText('hello')`.

```
$this->expectException(RuntimeException::class);  
$this->expectExceptionMessage('API key is missing');  
  
app(EmbeddingService::class)->embedText('hello');
```

Figure 12: Verifying missing API credentials and unsupported provider configurations

Similar validation tests were implemented for the language model service to verify missing API credentials and unsupported provider configurations.

Test Objectives

- Verify configuration validation.
- Confirm controlled exception handling.
- Prevent unnecessary external requests.
- Ensure predictable application behaviour.

Expected Outcome

Configuration errors are detected immediately, informative exceptions are generated, and the application fails safely without initiating invalid API requests.

6.3. Load Testing

Load testing was performed to evaluate the responsiveness, stability, and reliability of the AI-powered document retrieval system under concurrent workloads. The objective was to verify that background processing, semantic retrieval, database operations, and AI-assisted question answering continued to function correctly while multiple operations were executed simultaneously.

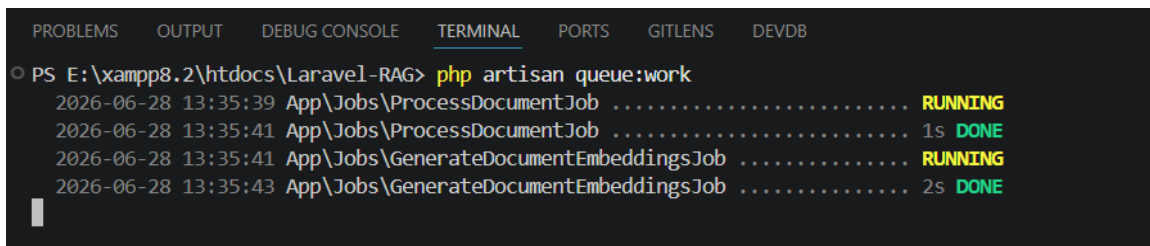
The evaluation focused on practical usage scenarios, including concurrent document uploads, repeated question-answering requests, semantic retrieval, and background processing. These tests helped identify potential performance bottlenecks and verify that the application remained responsive under the expected project workload.

The following sections summarize the load testing activities performed during the project.

6.3.1. Concurrent Document Processing

The document processing pipeline was tested by uploading multiple documents consecutively while monitoring background job execution and application responsiveness.

Document processing tasks, including text extraction, document chunking, embedding generation, and semantic indexing, were executed asynchronously through Laravel's queue system to prevent long-running operations from blocking user interaction.

A screenshot of a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), 'PORTS', 'GITLENS', and 'DEVDB'. The terminal shows a command prompt 'PS E:\xampp8.2\htdocs\Laravel-RAG>' followed by the command 'php artisan queue:work'. Below this, there are four lines of output showing the progress of background jobs. The first line is '2026-06-28 13:35:39 App\Jobs\ProcessDocumentJob RUNNING' in yellow. The second line is '2026-06-28 13:35:41 App\Jobs\ProcessDocumentJob 1s DONE' in green. The third line is '2026-06-28 13:35:41 App\Jobs\GenerateDocumentEmbeddingsJob RUNNING' in yellow. The fourth line is '2026-06-28 13:35:43 App\Jobs\GenerateDocumentEmbeddingsJob 2s DONE' in green. A white cursor is visible on the line following the fourth output line.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS DEVDB
PS E:\xampp8.2\htdocs\Laravel-RAG> php artisan queue:work
2026-06-28 13:35:39 App\Jobs\ProcessDocumentJob ..... RUNNING
2026-06-28 13:35:41 App\Jobs\ProcessDocumentJob ..... 1s DONE
2026-06-28 13:35:41 App\Jobs\GenerateDocumentEmbeddingsJob ..... RUNNING
2026-06-28 13:35:43 App\Jobs\GenerateDocumentEmbeddingsJob ..... 2s DONE
```

Figure 13: Laravel queue worker processing documents during concurrent uploads

Test Objectives

- Verify successful processing of multiple uploaded documents.
- Confirm correct execution of queued background jobs.
- Validate document status updates throughout processing.
- Ensure that the user interface remains responsive during processing.
- Verify successful completion of document indexing.

Expected Outcome

The application processes multiple uploaded documents sequentially through the queue system while maintaining a responsive user interface. Each document progresses through the expected processing stages until semantic indexing is completed successfully.

6.3.2. Concurrent Question Answering

Load testing also evaluated the system's ability to process multiple consecutive question-answering requests without affecting retrieval quality or application stability.

Multiple queries were submitted against different processed document collections while observing retrieval performance, response generation time, and overall responsiveness.

Test Objectives

- Verify stable processing of consecutive user queries.
- Confirm successful semantic retrieval for each request.
- Validate generation of context-aware responses.
- Ensure accurate citation generation.
- Maintain consistent application responsiveness.

Expected Outcome

The system successfully retrieves relevant document chunks, generates appropriate responses, and maintains stable performance during repeated conversational interactions.

6.3.3. Database Performance During Retrieval

Database performance was evaluated during repeated semantic retrieval requests to verify efficient similarity search and retrieval of indexed document content.

Testing focused on the PostgreSQL database and the pgvector extension to ensure that retrieval performance remained responsive while accessing larger document collections.

Test Objectives

- Verify efficient semantic retrieval.
- Confirm correct vector similarity search.
- Validate retrieval from indexed document collections.
- Ensure stable database performance during repeated searches.

Expected Outcome

Semantic retrieval remains responsive while returning relevant document chunks efficiently, demonstrating that the database structure and vector indexing strategy support the expected application workload.

6.3.4. Overall System Stability

The final stage of load testing evaluated the complete application while multiple software components operated simultaneously, including document uploads, background processing, database operations, semantic retrieval, and AI-assisted response generation.

The application remained responsive throughout testing, and all major workflows completed successfully without unexpected failures or inconsistent processing states. Although the evaluation was performed within the development environment, the observed results demonstrated that the implemented architecture is suitable for the intended project scope.

Test Objectives

- Verify stable operation under concurrent workloads.
- Confirm successful interaction between software components.
- Validate queue stability.
- Ensure reliable database operations.
- Verify consistent application behaviour throughout testing.

Expected Outcome

The application maintains stable operation while processing multiple concurrent activities, demonstrating that the implemented architecture is suitable for the expected operational workload.

6.4. Test Cases

Functional test cases were executed to verify that the implemented features of the AI-powered document retrieval system satisfied the specified functional requirements under realistic operating conditions. Unlike unit testing, which validates individual software components, functional testing evaluates complete application workflows from the user's perspective by verifying interactions between multiple system components.

Each test case was executed after the corresponding feature had been fully implemented and integrated into the application. The testing process covered both normal operating scenarios and exceptional conditions to ensure that document management, background processing, semantic retrieval, and AI-assisted question answering operated reliably under expected usage conditions.

The following test cases summarize the principal functional tests performed during the project.

TC01: Document Upload

Description

This test verifies that an authenticated user can successfully upload a supported document and that the document is accepted by the system for background processing.

Preconditions

- The user must be authenticated.
- The document upload page must be accessible.
- The Laravel queue worker must be running.
- The uploaded document must be one of the supported formats.

Steps

1. Open the Documents page.
2. Click Upload Document.
3. Enter the document title.
4. Select a supported document.
5. Click Upload.
6. Verify that the document appears in the document list.

Test Data

Condition	Expected Outcome
Valid PDF document	Document uploaded successfully
Valid DOCX document	Document uploaded successfully
Valid XLSX document	Document uploaded successfully
Unsupported file format	Validation error displayed
Missing document title	Validation error displayed

Table 1: TC01 – Document Upload Test Data

Expected Results

- The document is uploaded successfully.
- A new document record is created.
- The document enters the background processing queue.
- The processing status is displayed to the user.

Actual Results

- The document was uploaded successfully.
- The document record was created correctly.
- Background processing started automatically.
- The document status changed to Queued.

Postconditions

- The uploaded document is available for background processing.
- The document appears in the user's document repository.

Status: Passed

TC02: Document Processing Pipeline

Description

This test verifies that uploaded documents progress successfully through the complete document processing pipeline, including text extraction, document chunking, embedding generation, and semantic indexing.

Preconditions

- A supported document has been uploaded.
- The queue worker is running.
- External AI services are properly configured.

Steps

1. Upload a supported document.
2. Monitor the document processing status.
3. Wait for processing to complete.
4. Verify that the document status changes to Completed.

Test Data

Condition	Expected Outcome
PDF document	Processing completed successfully
Word document	Processing completed successfully
Excel document	Processing completed successfully
Missing AI configuration	Processing fails with appropriate error

Table 2: TC02 – Document Processing Pipeline Test Data**Expected Results**

- Text extraction completes successfully.
- Document chunks are generated.
- Embeddings are generated.
- Semantic indexing completes successfully.
- The document status changes to Completed.

Actual Results

- All processing stages completed successfully.
- The document became available for semantic retrieval.
- Processing status was updated correctly throughout the workflow.

Postconditions

- The processed document is available for question answering.

Status: Passed

TC03: AI-Assisted Question Answering**Description**

This test verifies that the system retrieves relevant document information and generates context-aware responses for user questions.

Preconditions

- At least one processed document exists.
- A conversation has been created.
- External AI services are available.

Steps

1. Create a new conversation.
2. Select one or more processed documents.
3. Enter a question.
4. Submit the question.
5. Review the generated response.

Test Data

Condition	Expected Outcome
-----------	------------------

Question related to uploaded document	Relevant response generated
Question outside document scope	Appropriate response generated based on the context
Follow-up question	Conversation context preserved

Table 3: TC03 – AI-Assisted Question Answering Test Data

Expected Results

- Relevant document chunks are retrieved.
- A context-aware response is generated.
- Supporting document references are included.
- The response is stored in the conversation history.

Actual Results

- Relevant information was successfully retrieved.
- The generated response reflected the uploaded document content.
- Document references were displayed correctly.
- The conversation history was updated successfully.

Postconditions

- The conversation contains both the user question and the generated response.

Status: Passed

TC04: Multi-Document Retrieval

Description

This test verifies that the system can retrieve relevant information from multiple selected documents and generate a unified response.

Preconditions

- Multiple processed documents exist.
- The selected documents contain related information.

Steps

1. Create a new conversation.
2. Select multiple processed documents.
3. Submit a question requiring information from several documents.
4. Review the generated response.

Test Data

Condition	Expected Outcome
Two related documents selected	Combined response generated
Three or more documents selected	Relevant information retrieved from multiple sources
Single document selected	Response generated using only the selected document

Table 4: TC04 – Multi-Document Retrieval Test Data

Expected Results

- Relevant information is retrieved from the selected documents.
- A consolidated response is generated.
- Appropriate document references are displayed.

Actual Results

- Information was successfully retrieved from multiple documents.
- The generated response accurately combined relevant document content.
- Supporting references were presented correctly.

Postconditions

- Conversation history contains the generated response.

Status: Passed

TC05: Conversation History**Description**

This test verifies that previous conversations remain available and that follow-up questions continue within the same conversation context.

Preconditions

- An existing conversation containing previous messages is available.

Steps

1. Open an existing conversation.
2. Review previous questions and responses.
3. Submit a follow-up question.
4. Verify that the conversation is updated.

Test Data

Condition	Expected Outcome
Existing conversation	Previous messages displayed
Follow-up question	Conversation updated correctly

Table 5: TC05 – Conversation History Test Data

Expected Results

- Previous conversation history is displayed.
- Follow-up questions are stored correctly.
- New responses are added to the existing conversation.

Actual Results

- Conversation history was displayed successfully.
- Follow-up interaction remained within the same conversation.
- The new response was stored correctly.

Postconditions

- The conversation contains the complete interaction history.

Status: Passed

TC06: Document Deletion

Description

This test verifies that users can successfully remove uploaded documents and that the deleted documents are no longer available within the system.

Preconditions

- A document exists in the user's repository.
- The user is authenticated.

Steps

1. Open the Documents page.
2. Select an existing document.
3. Click Delete.
4. Confirm the deletion.
5. Verify that the document is removed.

Test Data

Condition	Expected Outcome
Existing document	Document deleted successfully
Previously deleted document	Appropriate error or unavailable record

Table 6: TC06 – Document Deletion Test Data

Expected Results

- The selected document is deleted successfully.
- The document no longer appears in the document list.
- Associated processed information is removed from the system.

Actual Results

- The document was deleted successfully.
- The document was removed from the repository.
- Associated records were removed correctly.

Postconditions

- The deleted document is no longer accessible.

Status: Passed

7. Validation

Validation was performed to determine whether the developed AI-powered document retrieval system satisfied its intended user needs and project objectives under realistic usage conditions. While the previous chapter focused on verifying the technical correctness of individual components, validation assessed the complete system from the perspective of users and stakeholders.

The validation process covered the major application capabilities, including document management, automated document processing, semantic retrieval, AI-assisted question answering, conversation management, and administrative functionality. The objective was to confirm that these features worked together to provide an effective enterprise document retrieval solution.

Validation was conducted incrementally throughout development. Completed functionality was reviewed and evaluated against the defined requirements, allowing improvements to be incorporated before subsequent features were implemented.

The validation activities were organized into three main areas:

- **Sprint reviews** to evaluate development progress and completed functionality.
- **User story testing** to verify that the implemented features satisfied the defined user requirements.
- **Acceptance testing** to confirm that the complete system was suitable for its intended use.

The following sections describe these validation activities in detail.

7.1. Sprint Review

The AI-powered document retrieval system was developed incrementally through a series of development phases. At the end of each phase, the completed functionality was reviewed to evaluate development progress, verify expected behaviour, and identify improvements before continuing with subsequent work.

These sprint reviews provided continuous validation throughout the project by allowing completed features to be assessed as soon as they became available. This iterative approach helped identify issues early and ensured that newly implemented functionality remained consistent with the defined system requirements.

The major development phases are summarized below.

Sprint 1: Project Foundation and Authentication

Objective: Establish the initial application structure and provide secure user access.

Main Deliverables:

- Application foundation and database configuration.
- User authentication and account management.
- Protected routes for authenticated users.

Sprint Review Outcome:

The authentication workflow was reviewed by creating user accounts, testing login and password recovery, and verifying that unauthenticated users could not access protected functionality. The review confirmed that the application foundation was ready for implementing document management features.

Sprint 2: Document Management

Objective: Enable users to upload, view, and manage documents.

Main Deliverables:

- Document upload and management.
- Processing status indicators.
- Document validation and deletion.

Sprint Review Outcome:

The complete document management workflow was validated from upload through deletion. Users successfully uploaded supported documents, monitored processing progress, reviewed document information, and removed documents when required. Minor improvements were made to validation messages and status presentation.

Sprint 3: Document Processing Pipeline

Objective: Implement automated document processing for semantic retrieval.

Main Deliverables:

- Text extraction for supported document formats.
- OCR support for scanned PDFs.
- Text normalization and chunk generation.
- Metadata preservation.

Sprint Review Outcome:

Different document formats were processed to verify extraction quality, metadata preservation, and overall readability. The review resulted in improvements to text normalization, OCR handling, and metadata management before semantic indexing was introduced.

Sprint 4: Embedding Generation and Semantic Retrieval

Objective: Enable semantic search across processed documents.

Main Deliverables:

- Embedding generation.
- Vector database storage.
- Semantic similarity search.
- Retrieval filtering and ranking.

Sprint Review Outcome:

Natural-language queries were used to evaluate retrieval accuracy. The review confirmed that relevant document sections were retrieved based on semantic similarity while respecting the selected document collection. Minor refinements were made to retrieval limits and contextual information.

Sprint 5: Conversational Question Answering

Objective: Provide AI-assisted question answering based on retrieved document content.

Main Deliverables:

- Conversational interface.
- AI-assisted response generation.

- Source references.
- Conversation storage.

Sprint Review Outcome:

The complete question-answering workflow was evaluated using different document collections and query types. The review confirmed that generated responses remained grounded in the selected documents and that supporting references were presented correctly. Prompt construction and retrieval behaviour were refined based on the review.

Sprint 6: Multi-Document Retrieval and Conversation Management

Objective: Extend the conversational workflow to support multiple documents and persistent conversations.

Main Deliverables:

- Multi-document retrieval.
- Conversation history.
- Follow-up interactions.
- Context preservation.

Sprint Review Outcome:

The review validated realistic multi-document scenarios, including follow-up questions and reopening previous conversations. The system successfully preserved conversation history while restricting retrieval to the selected document collection.

Sprint 7: Background Processing and User Experience Improvements

Objective: Improve application responsiveness during document processing.

Main Deliverables:

- Asynchronous document processing.
- Live processing-status updates.
- Improved error handling and user feedback.

Sprint Review Outcome:

Multiple document uploads were executed while background workers processed documents asynchronously. The review confirmed that the user interface remained responsive and that processing status updates were displayed correctly. Error handling was also improved to provide clearer feedback for unsuccessful processing.

Sprint 8: Administration, Final Testing, and Deployment Preparation

Objective: Complete the application and prepare it for deployment.

Main Deliverables:

- Administrative functionality.
- Final code review.
- Testing and validation.

- Containerized deployment environment.
- Technical documentation.

Sprint Review Outcome:

The integrated application was reviewed by executing all major workflows from authentication through document retrieval and conversation management. The review confirmed that the principal functional requirements had been implemented successfully and that the application was ready for formal user story validation and acceptance testing.

Overall Sprint Review Result

The sprint review process provided continuous validation throughout the development lifecycle. Each major capability was reviewed before subsequent functionality was implemented, allowing issues to be identified and resolved early. By the final sprint, all major functional areas had been implemented, integrated, and successfully reviewed under realistic usage scenarios.

Overall, the sprint reviews confirmed that the system remained aligned with the project objectives and was ready to proceed with user story testing and acceptance testing.

7.2. User Story Testing

User story testing was performed to validate whether the implemented system satisfies the needs defined during the requirements specification phase. Unlike the functional test cases described in the previous chapter, which focused on specific system workflows and expected outputs, user story testing evaluated the application from the perspective of the intended users.

Each user story was validated by executing the corresponding functionality through the user interface and confirming that the expected user goal could be achieved successfully. The validation covered authentication, document management, document processing, conversational question answering, multi-document retrieval, source references, conversation history, and administrative functionality.

User Story 1: User Authentication

User Story: As a user, I want to securely register and log in so that I can access my documents and conversations.

Validation Activities:

- Registered a new account.
- Verified login with valid and invalid credentials.
- Confirmed that unauthenticated users could not access protected pages.

Validation Result: The authentication workflow operated as expected and protected restricted functionality from unauthenticated access.

Status: Passed

User Story 2: Uploading Documents

User Story: As a user, I want to upload supported documents so that their contents can be processed and used for knowledge retrieval.

Validation Activities:

- Uploaded supported PDF, Word, and spreadsheet documents.
- Verified successful upload and validation messages.
- Confirmed that uploaded documents appeared in the document list.
- Checked that background processing started automatically.

Validation Result: The system successfully accepted supported document formats and initiated the required processing workflow.

Status: Passed

User Story 3: Monitoring Document Processing Status

User Story: As a user, I want to monitor the processing status of uploaded documents so that I know when they are ready for question answering.

Validation Activities:

- Uploaded documents and observed their processing status.
- Verified status changes during extraction, chunking, embedding generation, and indexing.
- Confirmed that completed documents became available for conversational use.

Validation Result: The system displayed the processing progress correctly and informed users when documents were ready for retrieval.

Status: Passed

User Story 4: Managing Documents

User Story: As a user, I want to view and manage my uploaded documents so that I can maintain the document collection available for knowledge retrieval.

Validation Activities:

- Viewed the complete list of uploaded documents.
- Opened individual document details.
- Deleted an existing document.
- Confirmed that deleted documents were removed from the document list.

Validation Result: Users could successfully view, inspect, and delete their uploaded documents.

Status: Passed

User Story 5: Starting a Conversation

User Story: As a user, I want to create a new conversation and select one or more processed documents so that I can interact with their contents.

Validation Activities:

- Created a new conversation.

- Selected a single processed document.
- Selected multiple documents as knowledge sources.
- Confirmed that the created conversation appeared in the conversation history.

Validation Result: The system allowed users to create conversations and correctly associate selected documents with them.

Status: Passed

User Story 6: Asking Questions About Documents

User Story: As a user, I want to ask questions in natural language so that I can quickly retrieve relevant information from my uploaded documents.

Validation Activities:

- Submitted questions related to document content.
- Used questions that did not exactly match the wording of the source document.
- Verified that relevant information was retrieved.
- Confirmed that responses were generated from the selected document context.

Validation Result: The system successfully retrieved relevant document content and generated context-aware answers based on the selected documents.

Status: Passed

User Story 7: Multi-Document Query

User Story: As a user, I want to ask questions across multiple selected documents so that I can retrieve information from a combined knowledge source.

Validation Activities:

- Created a conversation with multiple selected documents.
- Asked questions related to content from different documents.
- Verified that retrieval was limited to the selected document collection.
- Confirmed that information from multiple sources could be used in the response.

Validation Result: The multi-document retrieval workflow operated correctly and returned information only from the selected document sources.

Status: Passed

User Story 8: Viewing Source References

User Story: As a user, I want to view source references with generated answers so that I can identify the origin of the retrieved information.

Validation Activities:

- Submitted questions with relevant information available in the selected documents.
- Verified that source references were displayed with the generated answers.
- Confirmed that the corresponding document source could be identified.

Validation Result: The system displayed relevant source information alongside generated answers, allowing users to identify the supporting documents.

Status: Passed

User Story 9: Viewing and Continuing Conversation History

User Story: As a user, I want to access previous conversations and continue asking follow-up questions so that I can preserve ongoing interactions with my documents.

Validation Activities:

- Created multiple conversations.
- Submitted several questions within the same conversation.
- Reopened a previous conversation from the history.
- Confirmed that earlier questions and answers were preserved.
- Continued the conversation with additional questions.

Validation Result: Conversation history was stored and restored correctly, allowing users to continue previous interactions without losing earlier content.

Status: Passed

User Story 10: Administrative Management

User Story: As an administrator, I want to monitor and manage system usage so that the application can be maintained effectively.

Validation Activities:

- Verified access restrictions for administrative functionality.
- Confirmed that regular users could not access administrative pages.
- Reviewed the available user and system management features.

Validation Result: Administrative functionality was restricted to authorized users and provided the intended management capabilities.

Status: Passed

Overall User Story Testing Result

The user story testing confirmed that all major user requirements were successfully implemented and could be completed through the application interface. Users were able to authenticate, manage documents, monitor processing, retrieve information through natural-language queries, continue conversations, and access administrative functionality where authorized.

Overall, the results demonstrated that the implemented system satisfied the intended user requirements and was ready for final acceptance testing.

7.3. Acceptance Testing

Acceptance testing was performed as the final validation activity to determine whether the complete AI-powered document retrieval system was suitable for its intended use and satisfied the overall project

objectives. While earlier testing activities focused on individual components, technical integrations, and specific user stories, acceptance testing evaluated the system as an integrated application.

The evaluation focused on realistic end-to-end workflows representing typical user interactions, including document management, semantic retrieval, AI-assisted question answering, conversation management, and administrative functionality.

The primary objective was to confirm that the complete user workflow could be executed without critical failures and that all major system components operated consistently together. The following acceptance scenarios were evaluated.

Acceptance Scenario 1: Secure Access to the Application

Acceptance Criteria:

- A user can register and log in successfully.
- Invalid login attempts are rejected.
- Unauthenticated users cannot access protected pages.
- Administrative functionality is restricted to authorized users.

Acceptance Result: The authentication and authorization mechanisms operated correctly. Users could access only the functionality permitted for their role, and protected pages remained inaccessible without valid authentication.

Status: Accepted

Acceptance Scenario 2: Uploading and Processing Documents

Acceptance Criteria:

- Users can upload supported document formats.
- Invalid or unsupported files are rejected with appropriate validation messages.
- Background processing starts automatically after upload.
- The processing status is visible to the user.
- Successfully processed documents become available for semantic retrieval.

Acceptance Result: The complete upload and processing workflow was successfully executed. Supported documents were accepted, processed through the required extraction and indexing stages, and made available for conversational interaction.

Status: Accepted

Acceptance Scenario 3: AI-Assisted Question Answering

Acceptance Criteria:

- Users can create a conversation and select processed documents.
- Natural-language questions can be submitted successfully.
- Relevant document content is retrieved.
- Generated responses are based on the selected document context.
- Source references are presented when supporting information is available.

Acceptance Result: The system successfully processed natural-language questions, retrieved relevant document content, and generated document-grounded responses. Supporting references were presented correctly, allowing users to identify the source of the information.

Status: Accepted

Acceptance Scenario 4: Multi-Document Information Retrieval

Acceptance Criteria:

- Users can select multiple processed documents as knowledge sources.
- Retrieval is restricted to the selected document collection.
- Relevant information can be retrieved from more than one selected document.
- A consolidated response can be generated from the retrieved context.

Acceptance Result: The multi-document workflow operated successfully. The system retrieved information from the selected document collection and generated responses using relevant content from multiple sources where applicable.

Status: Accepted

Acceptance Scenario 5: Conversation History and Follow-Up Interaction

Acceptance Criteria:

- Previous conversations remain available after navigation.
- Earlier questions and responses are preserved.
- Users can reopen previous conversations.
- Follow-up questions can continue within the existing conversation context.

Acceptance Result: Conversation history was stored and restored correctly. Users could return to previous conversations and continue asking follow-up questions without losing earlier interaction data.

Status: Accepted

Acceptance Scenario 6: Document Management

Acceptance Criteria:

- Users can view their uploaded documents.
- Document details and processing information are available.
- Users can delete documents when required.
- Deleted documents are no longer available for selection or retrieval.

Acceptance Result: The document management workflow operated correctly. Users could view, inspect, and delete documents, while the system maintained consistency between the document repository and conversational functionality.

Status: Accepted

Acceptance Scenario 7: Background Processing and Application Stability

Acceptance Criteria:

- Document processing runs asynchronously without blocking the user interface.
- Multiple queued jobs can be processed without leaving documents in inconsistent states.
- Processing status updates are visible.
- The application remains operational during repeated document and question-answering activities.

Acceptance Result: The system remained responsive while background jobs executed, and document processing states were updated consistently. Repeated application usage did not result in unexpected failures within the tested development environment.

Status: Accepted

Final Acceptance Result

The acceptance testing confirmed that the system satisfied the major functional objectives defined for the project. All primary user workflows, including document management, semantic retrieval, AI-assisted question answering, conversation management, and administrative functionality, operated successfully as an integrated application.

No critical functional issues were identified during acceptance testing. Based on the results, the system was considered suitable for its intended project scope and ready for final documentation and future deployment-related improvements.

8. Conclusion

The Software Verification and Validation activities carried out for the AI-powered document retrieval system demonstrated that the developed application satisfies its specified functional and quality requirements. Throughout the development process, verification techniques such as code reviews, integration testing, functional testing, and test case execution were applied to ensure that individual components operated correctly and that the complete system behaved as intended.

Validation activities, including sprint reviews, user story testing, and acceptance testing, confirmed that the implemented functionality met the project objectives and fulfilled the intended user requirements. The testing process verified the successful operation and integration of the system's major functional areas, while the iterative evaluation approach enabled issues to be identified and resolved throughout development.

Overall, the verification and validation process increased confidence in the correctness, reliability, and usability of the developed system. The structured testing approach demonstrated that the application is suitable for secure document management and intelligent knowledge retrieval while providing a solid foundation for future enhancements. Based on the completed verification and validation activities, the system is considered suitable for its intended project scope and ready for practical use.

9. References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. <https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-9780201633610>
- International Organization for Standardization. (2013). *ISO/IEC/IEEE 29119-1:2013 Software and systems engineering — Software testing — Part 1: Concepts and definitions*. ISO.
- Laravel. (2025). *Laravel Documentation*. <https://laravel.com/docs>
- Docker Inc. (2025). *Docker Documentation*. <https://docs.docker.com/>
- PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- pgvector Contributors. (2025). *pgvector: Open-source vector similarity search for PostgreSQL*. <https://github.com/pgvector/pgvector>
- PHPUnit Contributors. (2025). *PHPUnit Documentation*. <https://docs.phpunit.de/>
- Tesseract OCR Contributors. (2025). *Tesseract OCR User Manual*. <https://tesseract-ocr.github.io/>
- Poppler Developers. (2025). *Poppler PDF Rendering Library*. <https://poppler.freedesktop.org/>
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.